



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

Отчёт по лабораторной работе №2 по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ рекурсивного и нерекурсивного алгоритмов

Студент Попов Ю.А.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Постановка задачи	5
1.2 Рекурсия	5
1.3 Механизм выполнения рекурсивных вызовов	5
2 Конструкторская часть	6
2.1 Описание алгоритмов	6
2.2 Оценка трудоёмкости алгоритмов	7
2.2.1 Модель вычислений	7
2.2.2 Трудоёмкость итеративного алгоритма	9
2.2.3 Трудоёмкость рекурсивного алгоритма	10
2.3 Оценка затрачиваемой памяти	10
2.3.1 Итеративный алгоритм	10
2.3.2 Рекурсивный алгоритм	10
2.4 Используемые типы данных	11
2.5 Структура программы	11
3 Технологическая часть	12
3.1 Требование к программному обеспечению	12
3.2 Средства реализации	12
3.3 Реализация алгоритмов	12
3.4 Тестирование ПО	13
4 Исследовательская часть	14
4.1 Технические характеристики	14
4.2 Реализация замеров	14
4.3 Оценка времени работы алгоритмов	14
ЗАКЛЮЧЕНИЕ	17

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18
---	-----------

ВВЕДЕНИЕ

Целью данной работы является выполнить сравнительный анализ рекурсивного и нерекурсивного алгоритмов.

Для достижения цели поставлены следующие задачи:

- разработать рекурсивный и нерекурсивный алгоритмы решения задачи нахождения максимального числа последовательности;
- описать средства разработки и инструменты замера процессорного времени выполнения реализации алгоритмов;
- реализовать разработанные алгоритмы;
- выполнить тестирование реализации алгоритмов;
- выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти (для рекурсивного алгоритма — на материале анализа высоты дерева рекурсивных вызовов и оценки затрачиваемой на один вызов функции памяти);
- выполнить замеры процессорного времени выполнения реализации алгоритмов в зависимости от варьируемого входа;
- оценить трудоёмкость двух алгоритмов или их реализаций в худшем случае;
- сравнить результаты замеров процессорного времени и оценки трудоёмкости;
- сделать выводы из сравнительного анализа реализации рекурсивного и нерекурсивного алгоритмов решения одной и той же задачи, заданной вариантом, по критериям ёмкостной эффективности (на материале теоретической оценки пиковой временной эффективности).

1 Аналитическая часть

1.1 Постановка задачи

В лабораторной работе №2 необходимо провести сравнительный анализ рекурсивного и нерекурсивного алгоритма. По варианту необходимо найти максимальное число в последовательности чисел.

1.2 Рекурсия

Рекурсия — это такой способ организации вычислений, при котором процедура или функция в ходе выполнения обращается сама к себе. Другими словами, рекурсия является методом определения функций (процедур), при котором определяемая функция применена в теле своего же собственного определения [1].

Рекурсивный алгоритм — алгоритм, определяемый через себя [2].

1.3 Механизм выполнения рекурсивных вызовов

Каждый вызов подпрограммы в первую очередь создаёт в стеке стековый кадр (stack frame), или фрейм — блок данных, размещаемый на вершине программного стека. Помимо адреса возврата, он содержит, в частности, параметры и локальные переменные метода или ссылки на них, а также другую низкоуровневую информацию. Адрес возврата указывает на машинную команду, которой метод должен передать управление по завершении своих действий. По завершении метода его запись активации удаляется из программного стека, а освобождаемая память может использоваться для вызовов других методов. Таким образом, в любой момент программный стек содержит информацию только об "активных" подпрограммах, то есть вызванных, но ещё не завершённых (стековые кадры называют также активными кадрами, или записями активации) [3]. Глубина рекурсии определяется количеством вложенных вызовов и напрямую влияет на объем используемой памяти.

Вывод

В аналитической части рассмотрены основные понятия, связанные с рекурсией. Описана постановка задачи, проанализированы особенности рекурсивного подхода. Подчёркнута важность учёта глубины рекурсии при оценке трудоёмкости алгоритмов.

2 Конструкторская часть

В данной части будут реализованы блок-схемы алгоритмов поиска максимального числа в последовательности, произведена оценка их трудоёмкости и затрачиваемая память.

2.1 Описание алгоритмов

Оба алгоритма предназначены для нахождения максимального элемента последовательности. Входными данными служит массив целых чисел и его размер, содержащий минимум 1 элемент. Выходные данные: максимальный элемент массива.

На рисунке 2.1 изображена схема итеративного алгоритма. На рисунке 2.2 изображена схема рекурсивного алгоритма.

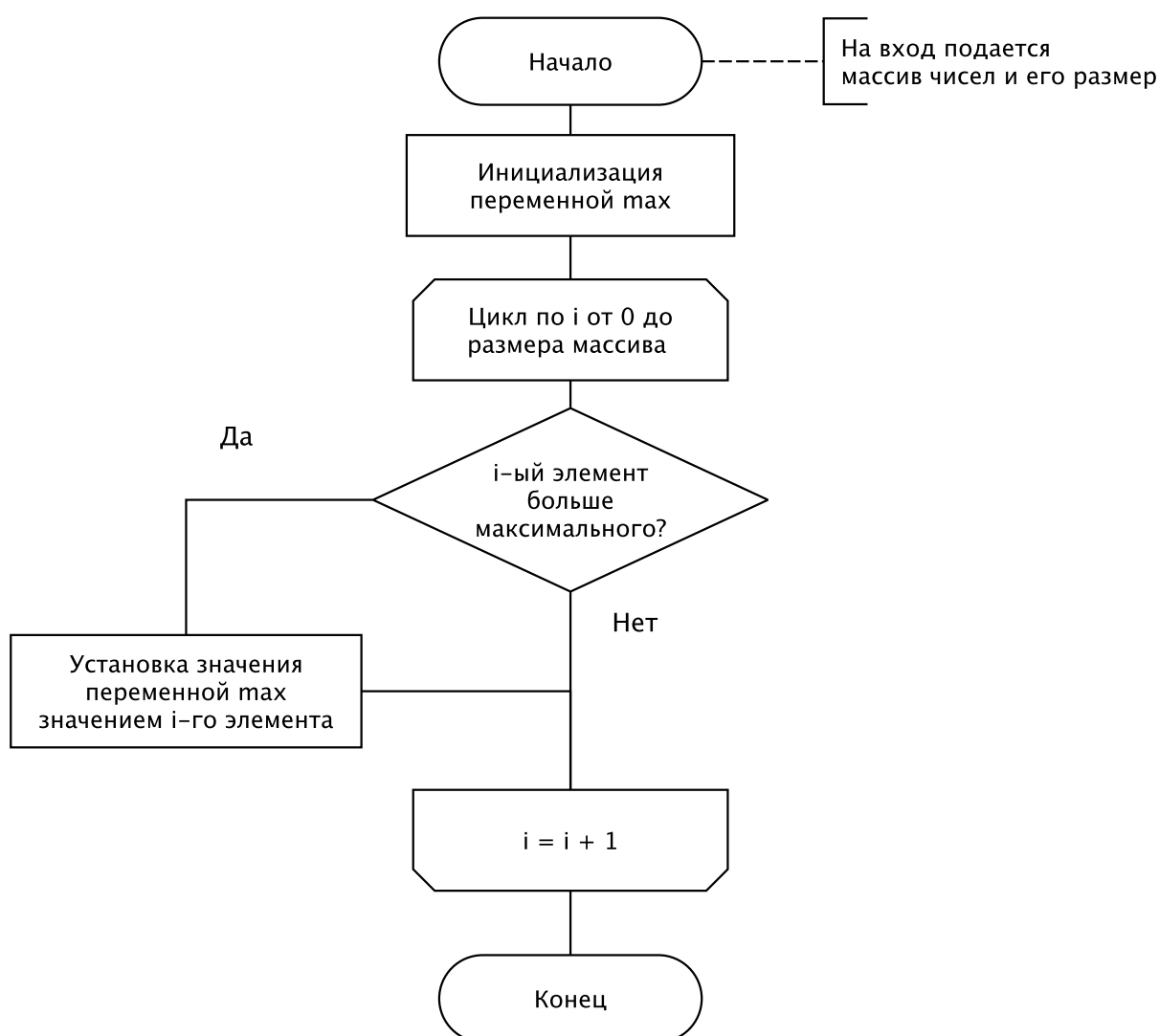


Рисунок 2.1 — Схема итерационного алгоритма нахождения максимального элемента массива

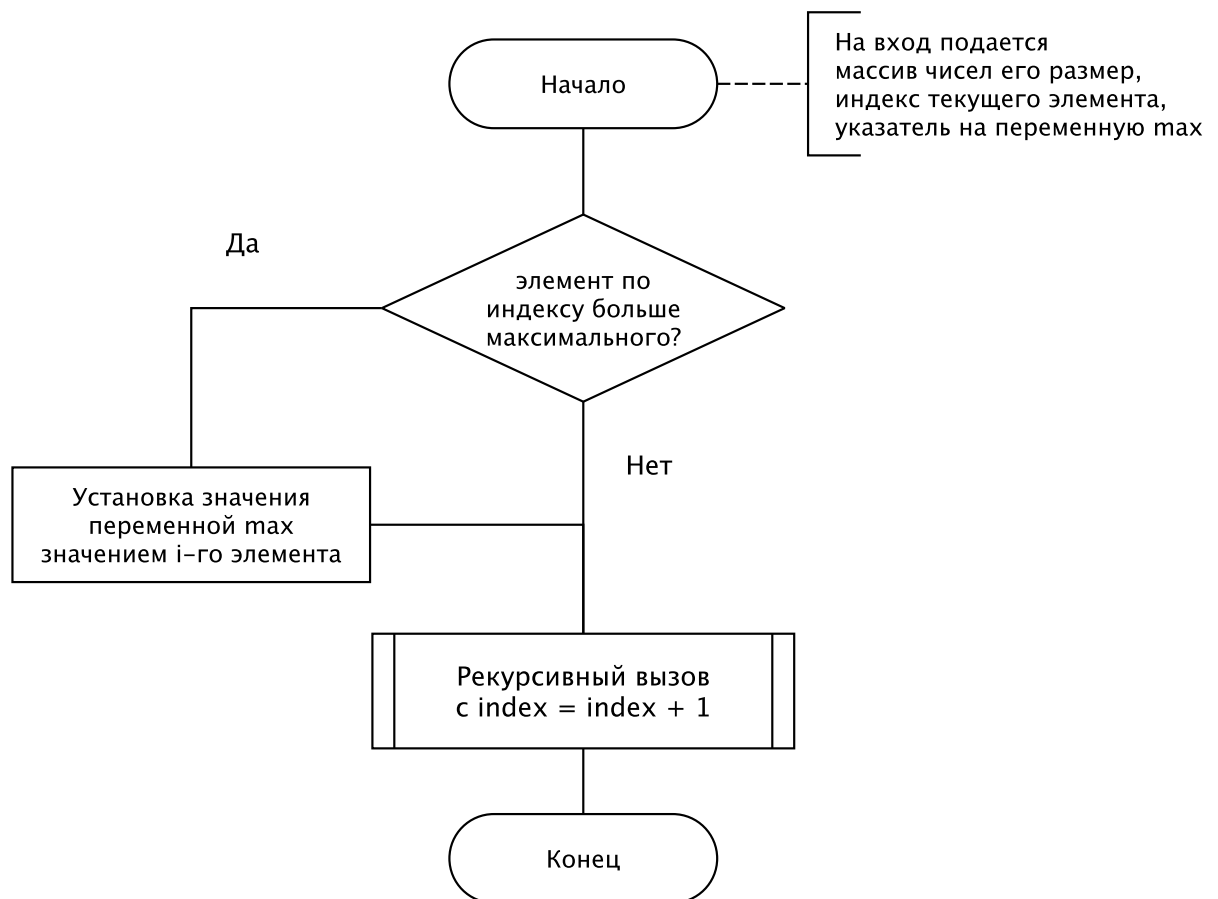


Рисунок 2.2 — Схема рекурсивного алгоритма нахождения максимального элемента массива

2.2 Оценка трудоёмкости алгоритмов

2.2.1 Модель вычислений

Была введена следующая модель вычислений:

— трудоёмкость следующих операций равной 1:

+, −, !, !=, ==, ≠, ≤, ≥, >, <, + =, − =, [], ||, &&, <<

— трудоёмкость следующих операция равной 2:

*, /, * =, / =, %

Трудоёмкость условного оператора определим:

$$f_{if} = f_{условия} + \begin{cases} \min(f_{true}, f_{false}), & \text{в лучшем случае} \\ \max(f_{true}, f_{false}), & \text{в худшем случае} \end{cases} \quad (2.1)$$

Трудоёмкость цикла определяется как:

$$f_{цикл} = f_{иниц.} + f_{усл.} + n \cdot (f_{тела} + f_{инк.} + f_{усл.}) \quad (2.2)$$

Для анализа трудоёмкости применяется метод, основанный на исследовании дерева вызовов рекурсии[4]. Согласно подходу, общая трудоёмкость рекурсивного алгоритма А на входных данных D складывается из компонент:

$$f = f_{R(D)} + f_{C(D)} \quad (2.3)$$

где:

- f — суммарная трудоёмкость алгоритма;
- $f_{R(D)}$ — затраты на организацию и обслуживание рекурсивных вызовов;
- $f_{C(D)}$ — трудоёмкость продуктивных вычислений.

Накладные расходы определяются количеством узлов в дереве рекурсии и стоимостью одного вызова:

$$f_{R(D)} = R(D) \cdot f_{R(1)} \quad (2.4)$$

где:

- $R(D)$ — число вершин дерева рекурсии;
- $f_{R(1)}$ — трудоёмкость входа в функцию и возврата.

Стоимость рекурсивного вызова и возврата оценивается по формуле:

$$f_{R(1)} = 2 \cdot (p + k + r + f + l + 1) \quad (2.5)$$

где:

- p — число передаваемых параметров;
- k — количество значение, возвращаемых по ссылке;
- r — число регистров процессора, сохраняемых в стеке;
- f — 1, если есть возврат, 0 если нету;
- i — количество локальных переменных.

Продуктивные вычисления делятся на действия, выполняемые во внутренних узлах дерева и действиях в листьях:

$$f_{C(D)} = f_{CV(D)} + f_{CL(D)} \quad (2.6)$$

где:

- $f_{CV(D)}$ — трудоёмкость во внутренних вершинах;
- $f_{CL(D)}$ — трудоёмкость в листьях.

Трудоёмкость в листьях рассчитывается следующим способом:

$$f_{CL(D)} = R_{L(D)} + f_{CL(1)} \quad (2.7)$$

где:

- $R_{L(D)}$ — число листьев дерева рекурсии;
- $f_{CL(1)}$ — вычислительная сложность при завершении рекурсии (обработка базового

случая).

В общем случае трудоёмкость во внутренних вершинах определяется суммированием по всему дереву:

$$f_{CV}(D) = \sum_{m=1}^{H_R(D)-1} \sum_{k=1}^{K(m)} f_{CV}(v_{mk}) \quad (2.8)$$

где:

- $H_R(D)$ — максимальная глубина рекурсии;
- $K(m)$ — количество внутренних вершин на уровне m ;
- v_{mk} — k -я вершина на этом уровне.

Подставив формулу 2.4, 2.7, 2.8 в выражение 2.3, получится общая формула трудоёмкости рекурсивного алгоритма:

$$f_A(D) = R(D) \cdot f_{R(1)} + R_L(D) \cdot f_{CL}(1) + \sum_{m=1}^{H_R(D)-1} \sum_{k=1}^{K(m)} f_{CV}(v_{mk}). \quad (2.9)$$

В частном случае, когда продуктивные вычисления одинаковы во всех узлах, формула упрощается. Пусть $R_V(D)$ — число внутренних вершин, а $f_{CV}(v)$ — постоянная трудоёмкость одного внутреннего узла. Тогда:

$$f_{CV(D)} = R_V(D) \cdot f_{CV(v)} \quad (2.10)$$

и в итоге, оценка трудоёмкости принимает вид:

$$f_A(D) = R(D) \cdot f_{R(1)} + R_L(D) \cdot f_{CL}(1) + R_V(D) \cdot f_{CV(v)}. \quad (2.11)$$

2.2.2 Трудоёмкость итеративного алгоритма

Выделяются два случая: в лучшем случае, обменов происходить не будет, и таким образом, трудоёмкость можно получить следующим способом:

$$f = f_{\text{инициализации}} + f_{\text{условия}} + n \cdot (f_{\text{тела}} + f_{\text{инкремента}} + f_{\text{условия}}) \quad (2.12)$$

$$f = 1 + 1 + n \cdot (2 + 1 + 1) \quad (2.13)$$

$$f = 2 + 4n \quad (2.14)$$

А в худшем случае:

$$f = f_{\text{инициализации}} + f_{\text{условия}} + n \cdot (f_{\text{тела}} + f_{\text{инкремента}} + f_{\text{условия}}) \quad (2.15)$$

$$f_{\text{тела}} = f_{\text{условия}} + f_{\text{тела_цикла}} = 4 \quad (2.16)$$

$$f = 1 + 1 + n \cdot (4 + 1 + 1) \quad (2.17)$$

$$f = 2 + 6n \quad (2.18)$$

2.2.3 Трудоемкость рекурсивного алгоритма

Для рекурсивного алгоритма трудоемкость обслуживания одного вызова определяется по формуле 2.5

$$f_{R(1)} = 2 \cdot (1 + 1 + 0 + 0 + 0 + 1) = 6 \quad (2.19)$$

Трудоемкость в листе дерева считается по формуле 2.20

$$f_{CL} = 1 \quad (2.20)$$

Трудоемкость во внутреннем узле дерева считается по формуле 2.21

$$f_{CV} = 5 \quad (2.21)$$

Общая трудоемкость рекурсивного алгоритма находится по формуле 2.22

$$f_{\text{рекурсии}} = 6 * N + 1 * 4 + (N - 1) * 5 = 11N - 1 \quad (2.22)$$

2.3 Оценка затрачиваемой памяти

2.3.1 Итеративный алгоритм

Итеративная реализация алгоритма не требует выделения дополнительной памяти, таким образом. Определим что, V это мера затрачиваемой памяти, тогда:

$$V_{\text{итерационного}} = 1 \quad (2.23)$$

2.3.2 Рекурсивный алгоритм

Рекурсивная реализация алгоритма хранит состояние каждого вызова в аппаратном стеке. Объем используемой памяти прямо-пропорционален глубине рекурсии. Размер одного кадра стека определяется количеством данных, сохраняемым при каждом вызове. Согласно методике[4], объем кадра вычисляется по формуле 2.24:

$$V_{\text{кадра}} = p + k + r + f + l + 1, \quad (2.24)$$

где:

— p — число передаваемых параметров;

- k — количество значений, возвращаемых по ссылке;
- r — число сохраняемых регистров процессора;
- $f = 1$, если функция, и $f = 0$ если процедура);
- l — количество локальных переменных;
- 1 — ячейка для хранения адреса возврата.

Подставляя значения в формулу (2.24), получим:

$$V_{\text{кадра}} = 1 + 1 + 1 + 0 + 0 + 1 = 4 \quad (2.25)$$

Во всех случаях рекурсия достигнет своей максимальной глубины, равной длине массива. Тогда объем дополнительной памяти будет равен произведению размера кадра на глубину рекурсии.

$$V_{\text{рекурсии}} = 4 * N \quad (2.26)$$

2.4 Используемые типы данных

При реализации алгоритмов использовались следующие типы данных:

- размер массива: `size_t`;
- тип данных массива: `int`;

2.5 Структура программы

Программное обеспечение будет состоять из нескольких частей:

- главный модуль — предоставляет возможность выбора режима работы программы, а также обеспечивает запуск программы;
- интерфейс — модуль для взаимодействия пользователя с программой. Пользователь может выбрать режим работы, а также ввести входные данные для поиска максимального элемента массива или тестовых замеров;
- модуль, содержащий функции для работы с массивами — предоставляет возможность объявить, проинициализировать, прочесть с терминала, и заполнить случайными числами;
- модуль для совершения замеров процессорного времени выполнения поиска максимального числа;

Вывод

На основе аналитической части были реализованы блок-схемы алгоритмов поиска максимального числа массива, произведена оценка их трудоёмкости и затрачиваемой памяти, а также продуманы модули программы.

3 Технологическая часть

В данной части будут описаны требования к реализации программного обеспечения, выбору языков программирования и тестированию.

3.1 Требование к программному обеспечению

Программное обеспечение должно удовлетворять следующим критериям:

- программа должна обеспечить возможность ввести последовательность для последующего нахождения максимума её элементов. Для успешного ввода пользователь должен иметь возможность задать длину последовательности и поэлементно ввести содержимое матрицы;
- для замеров времени выполнения, ПО должно обеспечить возможность ввода начального и конечного размера матриц, а также ввести шаг замеров;
- программа должна иметь возможность нахождения максимума двумя алгоритмами: итерационным и рекурсивным

3.2 Средства реализации

В качестве средства реализации был выбран язык C++[5], из-за возможности управления памятью и наличия контейнерных классов, встроенных в стандартную библиотеку. Разработка проводилась в среде программирования VS Code. Для сборки использована утилита CMake. Для визуализации данных, полученных в ходе исследования использовался язык Python[6] за счёт обилия библиотек для анализа данных. Мною были использованы pandas[7] и matplotlib[8]. Для запуска тестов и последующего сохранения результатов также использовался язык Python.

3.3 Реализация алгоритмов

При выполнении лабораторной работы были реализованы два варианта алгоритма поиска максимума. Реализация итерационного алгоритма указан на листинге 3.1. Реализация рекурсивного алгоритма указан на листинге 3.2

```
void Sequence::default_max(int &max) {
    max = INT_MIN;
    for (size_type i = 0; i < _len; i++) {
        if (_container[i] > max)
            max = _container[i];
    }
}
```

Листинг 3.1 — Итерационный алгоритм поиска максимума

```

void Sequence::recursion_max(int &max, size_type index) {
    if (index == _len)
        return;
    if (_container[index] > max)
        max = _container[index];
    recursion_max(max, index += 1);}

```

Листинг 3.2 — Рекурсивный алгоритм поиска максимума

3.4 Тестирование ПО

Для тестирования реализованы ПО были выделены следующие тестовые случаи для каждого алгоритма:

Таблица 3.1 — Тестовые случаи для функции рекурсивного поиска максимума

Входная последовательность	Ожидаемый максимум	Описание теста
{−1, −3, 0}	0	Максимум ноль
{−1, −3, −6}	−1	Максимум отрицательный
{100, 23, 6}	100	Максимум положительный
{100}	100	Последовательность из 1 элемента
{100, 6}	100	Последовательность из 2 элементов
{100, 50, 20, 40}	100	Максимум первый
{100, 50, 33, 200}	200	Максимум последний
{40, 20, 10, 0, 100, 50}	100	Максимум в середине

Все тесты прошли успешно.

Вывод

На основе конструкторской части были реализованы и протестированы алгоритмы нахождения максимума матрицы.

4 Исследовательская часть

В данной части будет проведено исследование процессорного времени выполнения разработанных алгоритмов.

4.1 Технические характеристики

Исследования проводились на устройстве со следующими характеристиками[3]

- операционная система: MacOS Tahoe 26.0.1 (25A362);
- процессор: Apple M3, 8 ядер;
- оперативная память: 16 Гигабайт, LPDDR5;

4.2 Реализация замеров

Во время выполнения замеров устройство было подключено к источнику питания и не выполняло сторонних процессов. Замеры проводились с помощью встроенной библиотеки *chrono*. Для каждого размера последовательности проводилось 500 запусков для каждого алгоритма, после чего время выполнения усреднялось. Для проведения замеров была отключена оптимизация компилятора флагом `-O0`[9]. Реализация функции замера времени выполнения приведена на листинге 4.1

```
auto start = std::chrono::high_resolution_clock::now();
for (size_t j = 0; j < EXP_COUNT; j++) {
    max = INT_MIN;
    sequence->recursion_max(max, 0);
}
auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(
    end-start);
```

Листинг 4.1 — Функция замера времени выполнения

4.3 Оценка времени работы алгоритмов

Были проведены замеры процессорного времени для реализаций алгоритмов нахождения максимума последовательности с размерами от 0 до 35000. Шаг увеличения размеров для обоих замеров — 1000.

На рисунках 4.1, 4.2, 4.3 продемонстрирована зависимость времени выполнения в зависимости от размера и содержимого последовательности. Итерационная реализация алгоритма в худшем случае выполняется быстрее в 4.4 раза (рисунок №4.2) чем рекурсивная, а в лучшем случае в 7 раз (рисунок №4.1). Скорость выполнения реализации алгоритма зависит не только от размера массива, но и от самих данных, которые содержатся в массиве. Если числа отсортированы в порядке возрастания, то каждую итерацию будет обновляться максимум. Напротив,

если последовательность отсортирована по убыванию, то значение максимума установится, обновится только 1 раз. Таким образом лучший случай выполняется быстрее в среднем 1.2 раза чем худший.

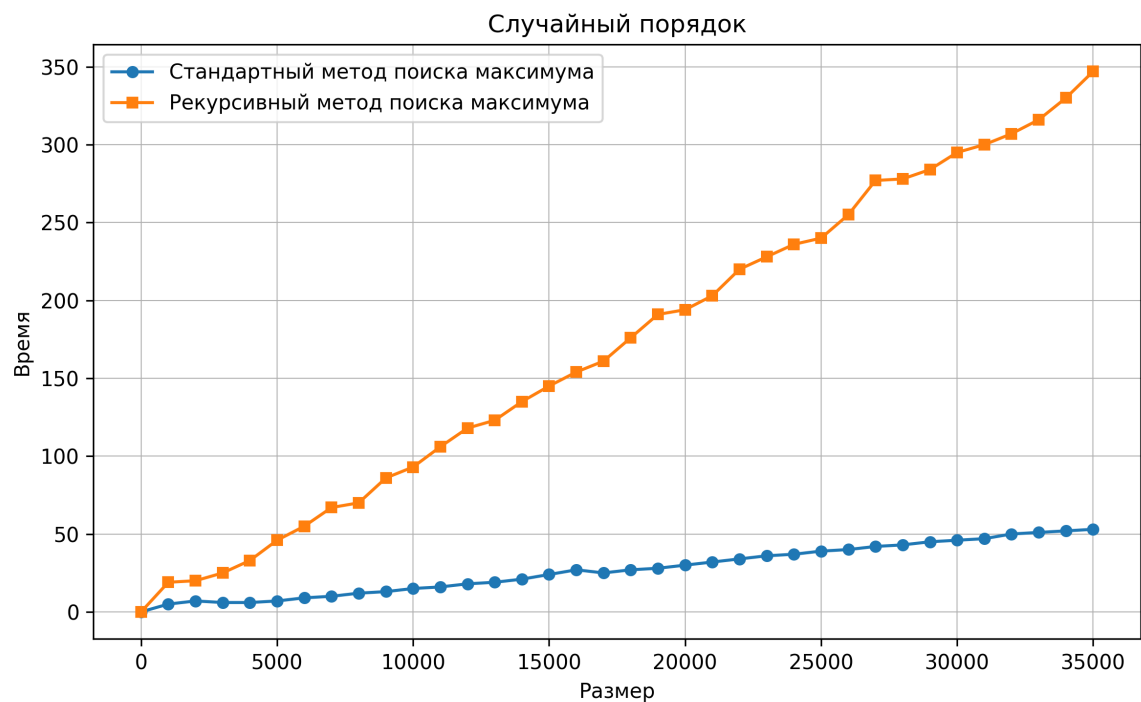


Рисунок 4.1 — Зависимость времени выполнения реализации алгоритма от размера массива, заполненного случайными числами

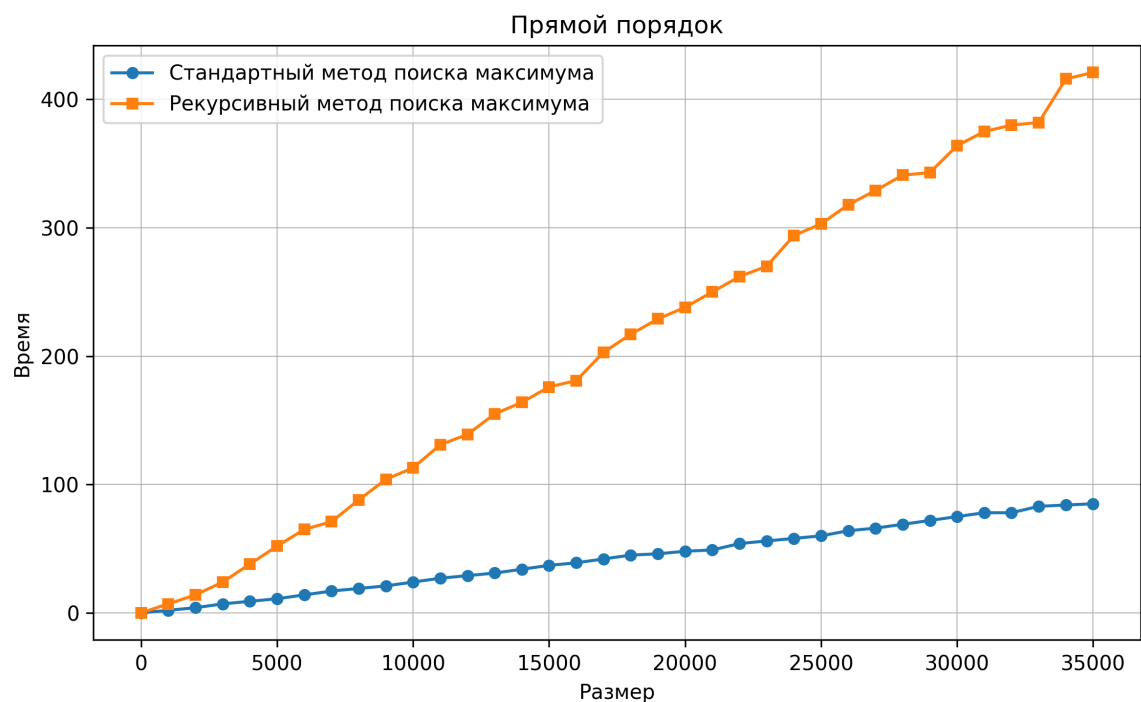


Рисунок 4.2 — Зависимость времени выполнения реализации алгоритма от размера массива, заполненного числами в порядке возрастания

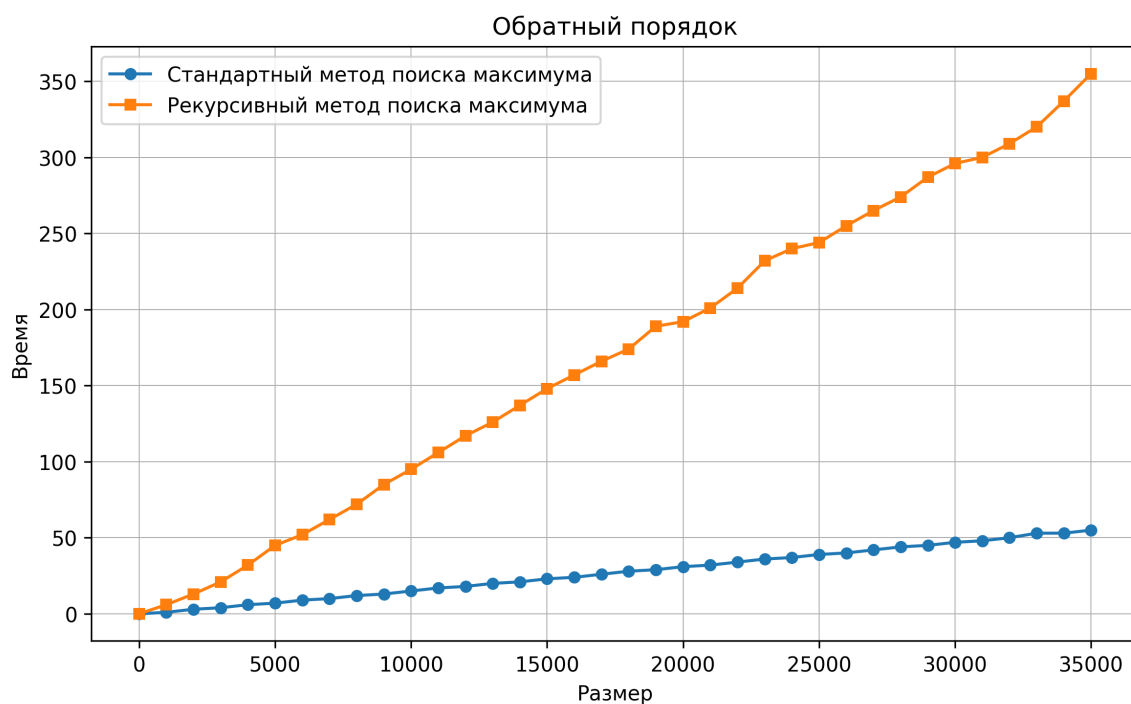


Рисунок 4.3 — Зависимость времени выполнения реализации алгоритма от размера массива, заполненного числами в порядке убывания

Вывод

реализации исследовательской части было выяснено, что итерационный алгоритм поиска максимально элемента массива выполняется быстрее в 7 раз в лучшем случае и в 4 раза в худшем случае чем рекурсивный алгоритм. Была установлена зависимость времени выполнения от сортировки входных данных. Выяснено, что поиск максимального элемента в массиве отсортированном в порядке убывания оказался быстрее на 20% чем поиск в массиве отсортированном в порядке возрастания.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №2 были выполнены следующие задачи:

- 1) разработан рекурсивный и нерекурсивный алгоритмы решения задачи нахождения максимального числа последовательности;
- 2) описаны средства разработки и инструменты замера процессорного времени выполнения реализации алгоритмов;
- 3) реализованы разработанные алгоритмы;
- 4) произведено тестирование реализации алгоритмов;
- 5) выполнена теоретическая оценка затрачиваемой реализацией каждого алгоритма памяти;
- 6) выполнены замеры процессорного времени выполнения реализации алгоритмов в зависимости от варьируемого входа;
- 7) оценена трудоёмкость двух алгоритмов или их реализаций в худшем случае;
- 8) сравнены результаты замеров процессорного времени и оценки трудоёмкости;
- 9) сделаны выводы из сравнительного анализа реализации рекурсивного и нерекурсивного алгоритмов решения одной и той же задачи, заданной вариантом, по критериям ёмкостной эффективности (на материале теоретической оценки пиковой временной эффективности).

В ходе лабораторной работы было выяснено, что итерационный алгоритм поиска максимально элемента массива выполняется быстрее в 7 раз в лучшем случае и в 4 раза в худшем случае чем рекурсивный алгоритм. К тому же итерационный алгоритм использует меньше памяти, потому что не происходит множественного вызова рекурсии. Кроме этого, рекурсивный алгоритм не получится использовать на массиве большого объёма по причине переполнения стека.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *РЕШЕНИЕ ЗАДАЧ С ИСПОЛЬЗОВАНИЕМ РЕКУРСИИ* [Электронный ресурс]. URL: <https://al.cs.msu.ru/files/recursia.pdf> (дата обращения: 26.10.2025).
2. *Рекурсия* [Электронный ресурс]. URL: <https://zns.susu.ru/IT/ВР/10.pdf> (дата обращения: 26.10.2025).
3. *Выполнение программы. Программный стек. Стековые кадры* [Электронный ресурс]. URL: https://it.kgsu.ru/Python_Recursion/pythonRecursion159.html (дата обращения: 26.10.2025).
4. М. В. Ульянов. РЕСУРСНО-ЭФФЕКТИВНЫЕ КОМПЬЮТЕРНЫЕ АЛГОРИТМЫ. РАЗРАБОТКА И АНАЛИЗ — Наука Физмалит 2007.
5. *Standard C++* [Электронный ресурс]. URL: <https://isocpp.org/> (дата обращения: 26.10.2025).
6. *Python* [Электронный ресурс]. URL: <https://www.python.org/> (дата обращения: 26.10.2025).
7. *Pandas* [Электронный ресурс]. URL: <https://pandas.pydata.org/> (дата обращения: 26.10.2025).
8. *Matplotlib — Visualization with Python* [Электронный ресурс]. URL: <https://matplotlib.org/> (Дата обращения: 26.10.2025).
9. *Clang command line argument reference* [Электронный ресурс]. URL: <https://clang.llvm.org/docs/ClangCommandLineReference.html#optimization-level> (Дата обращения: 27.10.2025).